

1. Github link: <https://github.com/psarkerbd/Simple-Calculator>

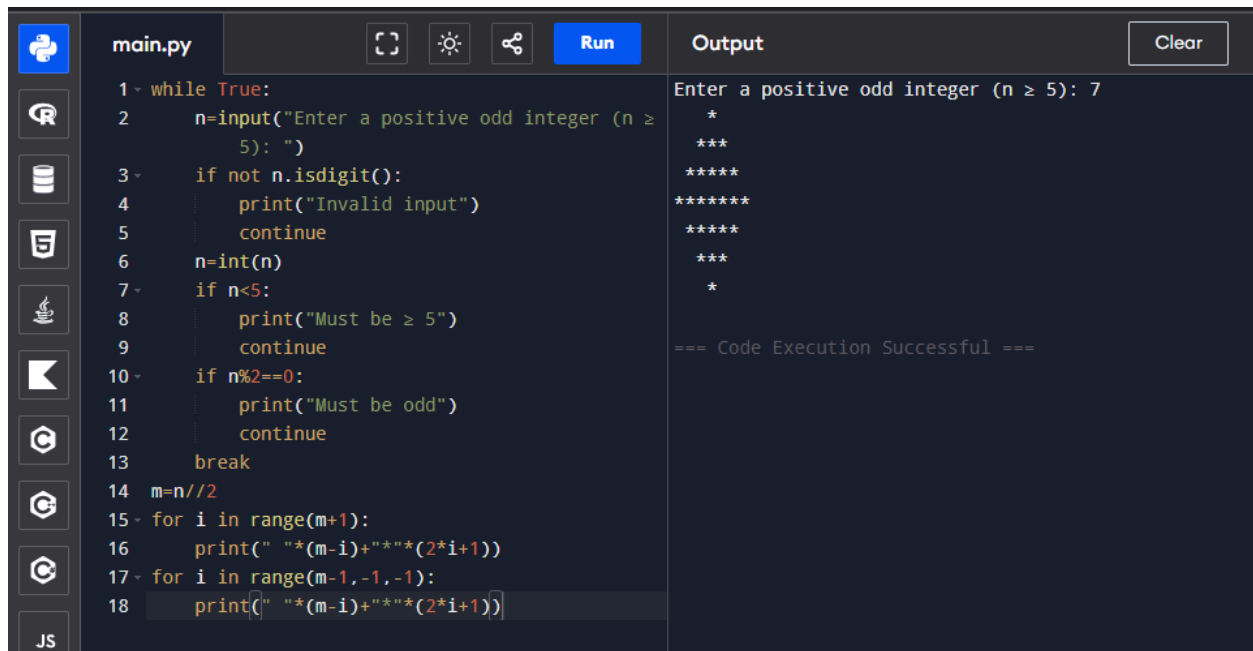
- a. This Program run on both Windows and Linux operating system by using pycharm, anaconda 3 or all other python editor with the python version 3.x
 - b. This software is developed by python 3.x
 - c. Used tkinter module for the Graphical User Interface
- (written on github readme file, asked chatgpt for help)

2. Pass usage:

The pass statement is used in coding when a function, class, or some part of the program is yet to be implemented. It is used to write the structure of the program first and then come back to implement it later. It is also used in collaborative coding, in which functions can be defined in advance for other developers to implement.

However, it is recommended to avoid using pass in cases where some actual code can be written. In addition, it is recommended to avoid using pass in cases where errors are expected, as it may simply ignore them. Also, it is recommended not to leave functions or classes empty and use pass. Instead, it is recommended to use `NotImplementedError`. In addition, it is recommended not to put pass in an infinite loop, such as `while True`, as it may cause an endless loop without producing any output.

2. 1.



The image shows a code editor interface with a dark theme. On the left is a sidebar with icons for Python, a debugger, a database, a file explorer, a terminal, and a JavaScript file. The main area is split into two panes. The left pane, titled 'main.py', contains the following Python code:

```
1 while True:
2     n=input("Enter a positive odd integer (n ≥
3     if not n.isdigit():
4         print("Invalid input")
5         continue
6     n=int(n)
7     if n<5:
8         print("Must be ≥ 5")
9         continue
10    if n%2==0:
11        print("Must be odd")
12        continue
13    break
14    m=n//2
15    for i in range(m+1):
16        print("  *(m-i)+"*(2*i+1))
17    for i in range(m-1,-1,-1):
18        print("  *(m-i)+"*(2*i+1))
```

The right pane, titled 'Output', shows the program's execution. It starts with the prompt 'Enter a positive odd integer (n ≥ 5): 7'. This is followed by a series of asterisks forming a diamond shape: a single '*' on the first line, three '*' on the second, five '*' on the third, seven '*' on the fourth, five '*' on the fifth, three '*' on the sixth, and a single '*' on the seventh. The output ends with '=== Code Execution Successful ==='. A 'Clear' button is located in the top right corner of the output pane.

2.2 for 9*9

	main.py	Output
	<pre>1 n=9 2 m=n//2 3 for i in range(m+1): 4 print(" "*(m-i)+"*(2*i+1)) 5 for i in range(m-1,-1,-1): 6 print(" "*(m-i)+"*(2*i+1))</pre>	<pre>* *** ***** ***** ***** ***** *** *</pre>

For any odd number:

	main.py	Output
	<pre>1 n=int(input('Enter an odd number: ')) 2 m=n//2 3 for i in range(m+1): 4 print(" "*(m-i)+"*(2*i+1)) 5 for i in range(m-1,-1,-1): 6 print(" "*(m-i)+"*(2*i+1))</pre>	<pre>Enter an odd number: 11 * *** ***** ***** ***** ***** ***** ***** ***** *** *</pre>

3. - Using `print(..., end="")` prevents automatic newlines, allowing continuous printing on the same line for patterns.
4. - `ceil()` is used to round up after division to ensure correct alignment of stars (*).
5. - Using `//` instead of `/` gives an integer directly, which also works for alignment.

For n=9

	main.py	Run	Output
	<pre>1 n=9 2 m=n//2 3 for i in range(m+1): 4 print(" *(m-i)+"*(2*i+1)) 5 for i in range(m-1,-1,-1): 6 print(" *(m-i)+"*(2*i+1))</pre>		<pre>* *** ***** ***** ***** ***** ***** *** *</pre>

For n=11:

	main.py	Run	Output
	<pre>1 n=int(input('Enter an odd number: ')) 2 m=n//2 3 for i in range(m+1): 4 print(" *(m-i)+"*(2*i+1)) 5 for i in range(m-1,-1,-1): 6 print(" *(m-i)+"*(2*i+1))</pre>		<pre>Enter an odd number: 11 * *** ***** ***** ***** ***** ***** ***** ***** ***** *** *</pre>

My laptop isn't turning on, therefore, I couldn't use my laptop and took screenshot from my IDE, I am sorry for not being able to use my own device.